

Laboratórne cvičenie 2

Základy jazyka C# a .NET

Pokročilé objektové technológie

100 minút • Visual Studio 2026 • .NET 10 • syntax C# 12

V pripravenom konzolovom systéme doplníte spracovanie zásielok a porovnáte správanie tried, štruktúr, rekordov a rozhraní. Cieľom je vedieť kód vysvetliť, upraviť a overiť. GitHub Copilot, internet aj Antigravity môžete používať na vysvetľovanie, návrh kódu, ladenie a testovanie.

Čo budete vedieť po cvičení

- Rozlíšiť kopírovanie hodnoty a referencie, použiť [Flags], record a with.
- Použiť init, private set, const, readonly, konštruktor, base, override a new.
- Zavolať implicitné, explicitné a predvolené členy rozhraní.
- Použiť indexery, params, lokálne funkcie, Swap<T>, ref, n-tice a dekonštrukciu.
- Doplniť switch výraz, rozširujúcu metódu a projekciu do anonymného typu.
- Skontrolovať návrh AI pomocou konkrétneho príkladu, testu a debuggeru.

Orientačný plán na 100 minút

Minúty	Blok	Kontrolný výsledok
0–10	Príprava prostredia	Build bez chyby, 11 zelených testov
10–20	U1 Hodnotové typy a príznaky	Dva vlastné experimenty v Program.cs
20–32	U2 Rekordy a vlastnosti	Deconstruct funguje, spolu 12 zelených
32–45	U3 Vozidlá a polymorfizmus	Náklady 6,00 a 3,00, spolu 16 zelených
45–55	U4 Rozhrania a prístup	Audit obsahuje presun, spolu 17 zelených
55–72	U5 Depo a metódy	Vyhľadanie, validácia, štatistiky, 26 zelených
72–82	U6 Dispečing a rozšírenia	Päť pravidiel a súčet 40, spolu 40 zelených
82–90	Upevnenie alebo rozšírenie	Vlastný test alebo dokončenie s pomocou
90–100	Vlastná kontrola	Spustenie aplikácie a otázky na porozumenie

Pracujte vlastným tempom. Riešenie sa neodovzdáva. Časy v pláne slúžia na orientáciu počas 100-minútového cvičenia. Postup prispôbajte svojim skúsenostiam; k rozpracovaným úlohám sa môžete vrátiť neskôr. Ak ste bez posunu, použite pomoc na stranách 10 a 11 alebo požiadajte vyučujúceho. Ak základ poznáte, pokračujte k rozšíreniam.

Odporúčaná cesta: U1–U6 a vlastné experimenty. Testy a otázky vám pomôžu overiť, čomu rozumiete. Rozšírenia si vyberte podľa záujmu a zostávajúceho času. Začnite v priečinku starter.

Príprava prostredia

Orientačne 0–10 minút • Najprv overte preklad a spustenie testov.

Otvorte pripravené riešenie

1. Pred cvičením majte vo Visual Studio Installer nainštalované Visual Studio 2026 a pracovnú záťaž **.NET desktop development**. Projekt cieli na .NET 10; jazyk je zámerne nastavený na C# 12 podľa prednášky. Pri prvom obnovení balíkov potrebujete internet.
2. Cez **File > Open > Project/Solution** otvorte `starter/SmartLogistics.slnx`. Súbor má príponu **slnx**. V Solution Explorer uvidíte Core, App a Tests.
3. Pravým kliknutím na `SmartLogistics.App` zvolte **Set as Startup Project**. Spustite **Build > Build Solution**, potom **Test > Test Explorer > Run All**.
4. Očakávajte **40 testovacích prípadov: 11 úspešných, 29 neúspešných**. Zlyhania s `NotImplementedException` označujú úlohy. Chyba prekladu alebo chýbajúci SDK je problém prostredia, riešte ho hneď.
5. Spustite aplikáciu cez **Ctrl+F5**. Zatiaľ oznámi **5 nedokončených blokov a 0 neočakávaných chýb**. Hotový U1 môžete skúmať už teraz.

V Test Exploreri vyhľadávajte `U2_`, `U3_` atď. Testy majú aj vlastnosti `Unit` a `Level`. Zelené testy pripravených častí majú `Level=Smoke`; dopĺňané správanie kontrolujú testy `Level=Core`.

Otvorte Copilot Chat

Prihláste sa GitHub účtom s prístupom ku Copilotu. Kliknite na odznak GitHub Copilot vpravo hore a zvolte **Open Chat**. Režim **Ask** použite na vysvetlenie, **Agent** na úpravu kódu. Ak nie je Agent dostupný, nechajte si navrhnúť kód v Ask a vložte ho ručne. Ponuka aj skratky závisia od nastavenia IDE.

Prompt pre jeden krok: „Rieš iba U5a v `Services/Depot.cs` v projekte starter. Zachovaj názvy a signatúry. Vráť prvý balík podľa kódu bez rozlišovania veľkosti písmen alebo null. Neupravuj dodané testy. Po úprave vysvetli Package? a over test `U5_StringIndexer`.“

Pred prijatím zmeny si prečítajte rozdiely. Skontrolujte súbor, parametre, návratový typ a podmienky. Najprv predpovedzte výsledok, potom spustite test. Skúste si vlastnými slovami vysvetliť jednu prijatú alebo opravenú zmenu AI. Poznámky si môžete robiť pre seba; nič neodovzdávate.

Terminál a riešenie problémov

Nasledujúce príkazy spúšťajte z priečinka **starter**. Fungujú aj pri práci v Rideri, VS Code alebo Antigravity.

```
dotnet --list-sdks
dotnet restore SmartLogistics.slnx
dotnet build SmartLogistics.slnx
dotnet test SmartLogistics.slnx --filter "Unit=U2"
dotnet run --project SmartLogistics.App -- --unit 2
```

Ak chýba .NET 10, doplňte SDK cez Visual Studio Installer. Ak nevidíte testy, skontrolujte preklad a obnovenie NuGet balíkov. Ak sú hneď všetky zelené, overte, či ste otvorili starter. Ak nefunguje AI, použite prednášku a pomoc na konci návodu; kód možno doplniť aj ručne.

U1 Hodnotové typy a príznaky

10–20 minút • Prednáška snímky 44–47, 54 a 57–58

Súbory: Core/Models/GeoCoordinate.cs, Core/Models/PackageFlags.cs a App/Program.cs. Cesty Core a App označujú projekty SmartLogistics.Core a SmartLogistics.App.

Preskúmajte pripravený kód

GeoCoordinate je readonly record struct. Pri priradení sa kopíruje hodnota. readonly chráni stav existujúcej hodnoty; výraz with vytvorí novú hodnotu. Typ nie je automaticky „vždy na zásobníku“: môže byť súčasťou objektu alebo po boxingu uložený na halde. Rozhodujúca je hodnotová sémantika.

Metóda DistanceTo je pripravená. Používa lokálnu aproximáciu vzdialenosti s korekciou zemepisnej dĺžky podľa priemernej šírky a zaokrúhlením na dve miesta. V tomto cvičení ju nemusíte implementovať. Nejde o navigačný algoritmus.

PackageFlags je tiež pripravený, aby celý projekt od začiatku šiel preložiť. Jednotlivé nezávislé príznaky majú hodnoty 1, 2, 4, 8, 16 a 32. Atribút [Flags] pomáha pri formátovaní kombinácií, správne hodnoty bitov však nezabezpečí za vás. Heavy je ručne zadáný príznak; jeho hodnota sa automaticky neodvodí z WeightKg.

Urobte dva experimenty v bloku U1

1. V Program.cs nájdite komentár **EXPERIMENT U1** pri súradniciach. Zaň vložte nasledujúci riadok. Pred spustením si premyslite, či sa zmení aj origin.

```
copy = copy with { Latitude = 48.0 };
```

2. Pri príznakoch doplňte pridanie Heavy a odobratie Fragile. Vypočítajte výsledné číslo ešte pred spustením.

```
flags |= PackageFlags.Heavy;  
flags &= ~PackageFlags.Fragile;
```

3. Spustite --unit 1 alebo celú aplikáciu. Pôvodná šírka ostane **49,2231**, kópia bude mať **48**. Zostanú **Express a Heavy**, číselne **10**.
4. Pozrite si rozdiel medzi flags == PackageFlags.None a flags.HasFlag(PackageFlags.None). HasFlag pri nulovom príznaku vracia true, preto neprítomnosť príznakov kontrolujte rovnosťou s None.

Kontrolný bod

Spustite testy U1_. Všetky štyri už prechádzajú; ich úlohou je overiť pripravené správanie. Vyskúšajte oba experimenty a vysvetlite si ich výsledok. Celkový počet zelených testov zostáva **11 z 40**.

Otázky pre seba: Čo sa pri priradení súradnice kopíruje? Prečo by samostatný príznak s hodnotou 3 kolidoval s hodnotami 1 a 2? Prečo sa pri bežnej enum hodnote bez kombinovania bitov môže používať postupnosť 0, 1, 2, 3?

Pomoc AI: „Vysvetli na tomto kóde rozdiel medzi hodnotou a referenciou. Potom mi daj otázku na bitové operátory; odpoveď ukáž až po mojom pokuse.“

U2 Rekordy a vlastnosti balíka

20–32 minút • Prednáška snímky 7–12, 18–20, 49–53 a 61–62

Súbory: Core/Models/Package.cs, Core/Legacy/LegacyPackage.cs, App/Program.cs.

Porovnajete dve reprezentácie dát

LegacyPackage je meniteľná trieda s ručne zapísanými vlastnosťami. Package je pripravený pozičný record class: kompilátor vytvorí vlastnosti s `init`, hodnotovú rovnosť, `ToString` a dekonštrukciu všetkých piatich pozičných členov. Názvy aj parametre Package zachovajte; legacy trieda je samostatná pomôcka na porovnanie.

`record class` je referenčný typ. Priradenie `alias = original` kopíruje referenciu na ten istý objekt. Výraz `with` vytvorí nový objekt so zmenenými vlastnosťami. Kopírovanie je plytké: prípadné referencie na meniteľné vnorené objekty by zostali spoločné. Rekord môže mať aj vlastnosti `so set`; samotné slovo `record` nezaručuje úplnú nemennosť.

Všimnite si `DefaultInsuranceRate`: je to `const decimal`, hodnota je známa pri preklade. `InsuranceFee` je vypočítavaná vlastnosť `s =>`. Pri cenách používame `decimal` a literály `10m` alebo `0.02m`. Oproti tomu `readonly field` možno inicializovať aj v konštruktoze; `readonly` referencia sama nezamkne obsah objektu, na ktorý odkazuje.

Doplňte vlastnú dekonštrukciu

1. V `Package.Deconstruct(out string trackingNumber, out decimal basePrice)` nahraďte `TODO`. Nastavte obidva výstupné parametre z rovnomenných vlastností.
2. Spustíte test `U2_Deconstruct` a blok aplikácie U2. Pri pôvodnej cene 10 a novej cene 15 ostáva pôvodný balík nezmenený; poistenie je 0,20.
3. Všimnite si dve rovnosti: `ReferenceEquals(original, alias)` je `true`, kým dva osobitne vytvorené rekordy s rovnakými dátami sú hodnotovo rovnaké. `with` má inú referenciu.
4. Za komentárom `EXPERIMENT U2` skúste `original.BasePrice = 99m;`. Preložte, prečítajte chybu o `init` vlastnosti a riadok znovu zakomentujte. Výsledné riešenie musí ísť preložiť.

Prompt: „Doplň iba telo dvojparametrovej metódy `Deconstruct` v `Package.cs`. Zachovaj `out` parametre a ostatný rekord. Vysvetli, prečo sa `var (code, price) = package` viaže na túto metódu.“

Kontrola po dokončení U2

Po dokončení U2 prechádzajú testy `U2_` a spolu máte **12 z 40** zelených prípadov. Rozlíšte priradenie referencie, hodnotovú rovnosť a kopírovanie cez `with`. Vysvetlite si aj výsledok pokusu so zápisom do `init` vlastnosti.

Ak stíhate: Nechajte si navrhnúť modernú podobu `LegacyPackage` v chate a porovnajete verejné API. Zmena `set` na `init` a pridanie hodnotovej rovnosti menia správanie, preto nejde iba o skrátenie zápisu. V základnej časti legacy súbor neprepisujte; bonus B2 opisuje samostatný pokus.

U3 Vozidlá a polymorfizmus

32–45 minút • Prednáška snímky 19 a 27–35

Súbory: Core/Vehicles/DeliveryVehicle.cs, DeliveryVan.cs, ElectricCargoBike.cs, DroneDelivery.cs.

Doplňte tri malé implementácie

DeliveryVehicle je abstraktný predok. Jeho virtuálna metóda CalculateCost počíta $2,00 + 0,60 \times \text{km}$. Metóda Deliver je abstraktná a potomkovia už majú pripravenú implementáciu. DeliveryVan(...) : DeliveryVehicle(...) ukazuje primárny konštruktor a odovzdanie argumentov predkovi.

1. **U3a Dodávka:** V DeliveryVan.CalculateCost zavolajte base.CalculateCost(distanceKm). Elektrická dodávka násobí výsledok **0.75m**, ostatná **1.15m**. Nepočítajte vlastný iný základ.
2. **U3b Bicykel:** V ElectricCargoBike.CalculateCost vráťte **1.50m + (decimal)distanceKm × 0.15m**. Trieda je už sealed; od nej nemožno dediť. Deliver je hotový.
3. **U3c Dron:** Doplňte telo pripravenej metódy public new string GetDiagnostics(). Výsledok musí začínať [**DRON ŠPECIFICKÉ**] a obsahovať kód dronu. Kľúčové slovo new zachovajte.

Prompt: „Doplň iba TODO U3a a U3b podľa uvedených vzorcov. Použi decimal. Zachovaj override a base. Potom samostatne doplň U3c s new a vysvetli výsledok volania cez typ predka.“

Predpovedzte výsledok a overte ho

```
var drone = new DroneDelivery("DRONE");  
DeliveryVehicle vehicle = drone;  
Console.WriteLine(drone.GetDiagnostics());  
Console.WriteLine(vehicle.GetDiagnostics());
```

Prvé volanie použije metódu dronu, druhé metódu predka. new skrýva zdedený člen a výber závisí od statického typu premennej. override pokračuje vo virtuálnom polymorfizme, takže výber implementácie závisí od skutočného typu objektu aj pri volaní cez predka. Tu je new zámerný experiment; pri návrhu bežného polymorfného API býva vhodný override.

V bloku U3 uvidíte pri 10 km **6,00** pre elektrickú dodávku a **3,00** pre bicykel. Pri neelektrickej dodávke test očakáva **9,20**. Dron má pripravený override výpočtu ceny: cez predka vráti **7,50**.

Krátky pokus s debuggerom

V DeliveryVan nastavte bod prerušenia na riadku s base.CalculateCost a spustíte aplikáciu cez **F5**. Prejdite riadok pomocou **F10**. V Locals alebo Watch skontrolujte distanceKm, IsElectric a vypočítaný základ **8,00**. Potom nechajte aplikáciu dobehnúť.

Kontrolný bod: Všetky U3_ testy sú zelené, spolu **16 z 40**. Vysvetlite si dva diagnostické výpisy a skontrolujte hodnotu, ktorú ste videli v debuggeri. Ak sa cena líši, pozrite vzorec predka a typ literálov.

U4 Rozhrania a prístup k členom

45–55 minút • Prednáška snímky 36–42 a vlastnosti na snímke 11

Súbory: Core/Models/TrackedPackage.cs, Core/Interfaces/, Core/Security/FleetSecurity.cs, App/Program.cs.

Doplňte explicitný audit

Trieda TrackedPackage implementuje dve rozhrania. GetStatus() je verejná implicitná implementácia ITrackable. GetAuditRecord() je pripravená explicitná implementácia ISecureAuditable, ktorej telo čaká na doplnenie.

Nahradte TODO reťazcom začínajúcim **[INTERNÝ AUDIT]**. Zahrňte Package.TrackingNumber a položky _auditTrail spojené pomocou string.Join. História musí obsahovať pôvodnú lokalitu aj neskorší presun. Zachovajte podpis bez public:

```
string ISecureAuditable.GetAuditRecord()
```

V bloku U4 sa lokalita zmení zo Žiliny na Martin cez UpdateLocation. Vlastnosť CurrentLocation má pripravené private set, aby volajúci použil túto metódu a aktualizoval aj históriu. readonly List<string> chráni referenciu na zoznam; metóda Add môže obsah zoznamu meniť.

Prompt: „Doplň iba telo explicitnej metódy ISecureAuditable.GetAuditRecord v TrackedPackage. Vráť značku [INTERNÝ AUDIT], kód balíka a string.Join nad históriou. Vysvetli rozdiel medzi explicitnou implementáciou a autorizáciou.“

Vyskúšajte dva spôsoby volania

```
ITrackable customerView = tracked;  
Console.WriteLine(customerView.Ping());  
ISecureAuditable auditView = tracked;  
Console.WriteLine(auditView.GetAuditRecord());
```

Ping má predvolenú implementáciu v rozhraní. Volajte ho cez premennú ITrackable. Explicitný audit sa tiež volá cez typ rozhrania. Každý, kto má tento objekt, ho môže pretypovať na ISecureAuditable: ide o tvar API, skutočné oprávnenia by musela kontrolovať ďalšia logika.

Overte prístupové modifikátory

Modifikátor	Kto má prístup k členu
public	Kód, ktorý má prístup k samotnému typu
private	Deklarujúci typ
protected	Deklarujúci typ a jeho potomkovia
internal	Kód v tom istom zostavení
protected internal	To isté zostavenie ALEBO potomkovia
private protected	Deklarujúci typ a potomkovia v tom istom zostavení

V App/Program.cs odkomentujte postupne dva pokusy z EXPERIMENT U4, napríklad zápis CurrentLocation a prístup k DepotEncryptionKey. Preložte a vysvetlite si dôvod chyby. Každý pokus potom zakomentujte. Core a App sú v tomto riešení dve samostatné zostavenia.

Kontrola po dokončení U4: U4_ testy prechádzajú, spolu 17 z 40. Vysvetlite default člen, explicitný audit a aspoň dva prístupové modifikátory. Žiadny pokus s chybou prekladu nesmie zostať aktívny.

U5 Depo a metódy

55–72 minút • Prednáška snímky 15–16, 21–22, 24–25, 41–42 a 60–62

Súbor: Core/Services/Depot.cs. Doplňte tri TODO v poradí U5a, U5b, U5c.

U5a Vyhľadanie zásielky podľa kódu

Reťazcový indexer `this[string trackingNumber]` vracia **prvý** balík so zhodným kódom, bez rozlišovania veľkosti písmen cez `StringComparison.OrdinalIgnoreCase`. Keď kód neexistuje, vráti **null**. Package? túto možnosť vyjadruje v type.

Môžete použiť `foreach` alebo `FirstOrDefault`. Celočíselný indexer s `get` aj `set` je pripravený. Neplatný index vyvolá výnimku zo zoznamu; neexistujúci reťazcový kód vracia `null`. Nejde o rovnaké správanie.

U5b Registrácia s lokálnymi funkciami

V `RegisterWithValidation` vytvorte dve lokálne funkcie:

- `IsPrefixValid(string code)` vidí `CodePrefix` a `minCodeLength`. Overí prefix cez `OrdinalIgnoreCase` a dĺžku kódu **aspoň minCodeLength**.
- `static IsWeightValid(double weight)` overí **weight > 0.01 a weight <= 100.0**. Statická lokálna funkcia nezachytáva okolité premenné ani stav inštancie. Tento zápis automaticky nezaručuje vyššiu rýchlosť.

Balík pridajte do `_packages` iba pri splnení oboch podmienok; vtedy vráťte `true`. Pri odmietnutí vráťte `false` a počet balíkov nemeňte. Hraničná hmotnosť **0,01 sa odmieta, 100 sa prijíma**. Kód s prefixom a dĺžkou presne rovnou limitu sa prijíma.

Metóda má pripravené `minCodeLength = 6`. Zavolajte ju raz s predvolenou hodnotou a raz pomenovaným argumentom `minCodeLength`: 7. Metóda `Add` s `params` je hotová a obsluhuje hromadné vloženie bez tejto doménovej validácie.

U5c Štatistiky a pomenovaná n-tica

`GetStatistics` musí vrátiť počet balíkov, súčet `BasePrice` a súčet `WeightKg` v pripravenom type (`int TotalCount`, `decimal TotalValue`, `double TotalWeightKg`). Prázdne depo vráti **(0, 0m, 0.0)**. Použite cyklus alebo `Sum`.

Prompt: „Doplň len U5a až U5c v `Depot.cs` podľa návodu. Zachovaj signatúry. Pre validáciu použi jednu lokálnu a jednu static lokálnu funkciu. Po každej časti spusti súvisiace U5 testy a vysvetli hraničné hodnoty.“

Preskúmajte pripravené generiká a iterovanie

V bloku U5 aplikácie najprv očakávajúce štatistiky **2 balíky, hodnota 30, hmotnosť 5 kg**. Registrácia kódu SK-ZA-1003 uspeje, prefix CZ-ZA sa odmieta. `IEnumerable<Package>` umožní `foreach`.

Pripravené `Swap<T>(ref T a, ref T b)` prehodí **odovzdané premenné**. Vyskúšajte ho aj s dvoma `int` premennými. Pri balíkoch z depa sa po `Swap` zmenia lokálne referencie `a` a `b`, ale položky v samotnom depu zostanú na svojich pozíciách.

Kontrolný bod: Všetky `U5_` testy prechádzajú, spolu **26 z 40**. Vysvetlite si rozdiel `int` a `string` indexera, výsledky hraníc validácie a dôvod, prečo `Swap` nepremenil poradie v depu. Ak meškáte, použite pomoc na strane 11.

U6 Dispečing a rozšírenia

72–82 minút • Prednáška snímky 23, 56 a 60–63

Súbory: Core/Services/DispatcherService.cs, Core/Extensions/PackageExtensions.cs, App/Program.cs.

U6a Doplníte switch výraz

Metóda Dispatch vracia pomenovanú n-ticu (string VehicleType, decimal FinalPrice). Kontrola neplatných vstupov je hotová: vzdialenosť musí byť konečná a nezáporná, hmotnosť konečná a kladná a cena nezáporná. Nulová vzdialenosť aj nulová cena sú platné. Neplatný vstup vyvolá príslušnú výnimku pred rozhodovaním.

Za validáciou nahradíte TODO výrazom switch nad (package.Flags, distanceKm, package.WeightKg). Pravidlá dodržte **v tomto poradí**. Použite presné názvy vozidiel z tabuľky.

Poradie	Podmienka	Vozidlo a cena
1	Má RequiresColdChain	Chladiarenská dodávka × 1.50m
2	Km <= 5, kg <= 8, bez Heavy a Oversized	Elektrobicykel × 0.90m
3	Kg > 20 ALEBO Heavy ALEBO Oversized	Ťažká nákladná dodávka × 1.35m
4	Km > 60	Diaľkový kamión × 1.25m
5	Všetky ostatné prípady	Štandardná dodávka × 1.00m

Vždy násobíte BasePrice. Vyberá sa prvá zhodná vetva. Chladený 30-kilový balík na 70 km preto patrí do chladiarenskej dodávky. Pre Flags použite HasFlag alebo bitovú kontrolu, nie rovnosť s jedinou enum hodnotou. Bicykel má prednosť aj pri kombinácii Express a Fragile, ak spĺňa jeho podmienky.

Prompt: „Implementuj iba U6a za pripravenou validáciou. Použi switch výraz nad n-ticou, relačné vzory a when. Pravidlá a presné reťazce prevezmi z komentára metódy. Neupravuj testy ani validáciu. Vysvetli, prečo ColdChain musí byť prvý.“

U6b Súčet expresných zásielok

Doplníte TotalExpressValue(this IEnumerable<Package> packages). Sčítajte BasePrice balíkov, ktoré majú Express aj v kombinácii s inými príznakmi. Prázdna kolekcia vráti **0m**. Použite foreach alebo Where(...).Sum(...).

this pri prvom parametri umožní packages.TotalExpressValue(). Je to syntax volania statickej metódy; privátne členy Package tým nesprístupníte. Pre použitie musí byť dostupný namespace s metódou.

Experiment s anonymným typom

V bloku U6 nájdite new { p.TrackingNumber, IsUrgent = ... }. Pridajte p.WeightKg a doplníte jeho výpis. Anonymný typ má vlastnosti len na čítanie a typ premennej odvodí var. Pri troch ukázkových balíkoch je expresná hodnota **40m**.

Kontrola po dokončení U6: Prechádzajú všetky testy U6_, spolu **40 z 40**. Aplikácia oznámi **0 nedokončených blokov a 0 chýb**. Predpovedzte a overte výsledok pre 5 km a 8 kg, pre 60 km a pre kombinovaný ColdChain a Heavy. Potom pokračujte ďalšou stranou.

Upevnenie vedomostí a vlastná kontrola

Zvyšný čas využite na vlastný pokus, dokončenie alebo overenie.

Vyberte cestu podľa svojho postupu

Ak ešte dokončujete základ: Použite pomoc na konci návodu a spustite testy bloku, ktorý práve riešite. Zamerajte sa na jednu chybu naraz. Ak niečomu nerozumiete, požiadajte AI alebo vyučujúceho o vysvetlenie. Pokračujte vlastným tempom aj vtedy, keď ostatní už riešia rozšírenia.

Ak základ funguje: Vyberte jednu z nasledujúcich úloh. Rozsiahlejšie zmeny dokončíte až po cvičení; na záver si nechajte čas na spustenie a vysvetlenie.

B1 Vlastný test a odhalenie chyby

Pridajte nový xUnit test do **nového súboru**, napríklad StudentTests.cs. Vyberte správanie, ktoré dodané testy ešte neukazujú: nulovú cenu platného balíka, zápornú nekonečnú vzdialenosť alebo cenu bicykla pri nulovej vzdialenosti. Najprv určte očakávaný výsledok, až potom použite AI.

Nechajte Copilot navrhnúť test s Arrange, Act, Assert. Skontrolujte očakávanú hodnotu podľa zadania. Overte, že test prejde. Potom dočasne zmeňte príslušnú implementáciu tak, aby bol výsledok chybný, a spustite **váš test**. Musí zlyhať. Zmenu vráťte a test spustite znova. Vysvetlite si, akú chybu test zachytil.

B2 Refaktoring a plytká kópia

Vytvorte samostatný typ ModernLegacyPackage ako record s init vlastnosťami. Zachovajte LegacyPackage na porovnanie. Ukážte, čo sa mení pri rovnosti a možnosti meniť cenu. Alebo vytvorte malý samostatný record so zoznamom: použite with a preskúmajte, či obe kópie zdieľajú zoznam. Produkčný Package nemeňte.

B3 Rozšírenie pravidiel dispečingu

Pridajte pravidlo pre dron: Express aj Fragile, najviac 25 km, najviac 4 kg, bez Heavy a Oversized; cena × 1.80m. Vložte ho **po bicykli a pred ťažkou dodávkou**. Dodané testy nechajte nezmenené a pridajte vlastné testy pre 25 km, 25,01 km a konflikt s chladením. Dron vyhráva napríklad na 10 km; v meste stále vyhráva bicykel.

Záverečné overenie

Overte bloky, ktoré ste dokončili, príslušnými testami a spustením aplikácie. Odstráňte aktívne pokusy spôsobujúce chybu prekladu. V dokončených metódach nahraďte NotImplementedException funkčným kódom. Rozpracované úlohy môžete dokončiť neskôr. Pri oprave môžete využiť AI; výslednému kódu by ste mali rozumieť.

Riešenie ani odpovede na otázky neodovzdávate. Svoj projekt si uložte, aby ste mohli pokračovať neskôr. Dôležité je rozumieť tomu, čo ste vyskúšali, a vedieť výsledok overiť. Ak máte vlastné testy, počet testovacích prípadov môže byť vyšší ako 40.

Otázky pre vlastnú kontrolu: Čo kopíruje priradenie record class? Čo robí with? Ako sa líši new a override? Prečo audit zavoláte cez rozhranie? Čo zachytáva lokálna funkcia? Ktorá vetva switch vyhrá pri súbehu podmienok? Prečo zelený test sám nepotvrďuje pochopenie kódu?

Pomoc pre rekordy vozidlá a rozhrania

Použite túto stranu, ak ste po dvoch minútach bez posunu. Najprv skúste nápovedu, potom krátky vzor. Po vložení ho vysvetlite vlastnými slovami a spustite testy bloku. Vzory dopĺňajú **telá už pripravených metód**; nevytvárajte duplicitné triedy ani metódy.

U2 Nastavenie out parametrov

Out parameter musí dostať hodnotu pred normálnym návratom metódy. Názov parametra sa líši od vlastnosti veľkým písmenom.

```
trackingNumber = TrackingNumber;  
basePrice = BasePrice;
```

V Program.cs použite `var (code, price) = original;`. Pre inú kombináciu piatich hodnôt existuje aj kompilátorom vytvorená pozičná dekonštrukcia.

U3 Výpočet ceny a diagnostika

U3a používa základ z predka. Pri 10 km je tento základ 8m, až potom použijete koeficient podľa pohonu.

```
decimal baseCost = base.CalculateCost(distanceKm);  
return IsElectric ? baseCost * 0.75m : baseCost * 1.15m;
```

U3b má vlastný vzorec. Pri násobení decimal a double potrebujete explicitný prevod.

```
return 1.50m + (decimal)distanceKm * 0.15m;
```

U3c je zámerné skrytie metódy. Podpis s new je pripravený; telo môže byť jednoduché.

```
return $"[DRON ŠPECIFICKÉ] Kód: {LicensePlate}";
```

Keby ste new nahradili override, volanie cez DeliveryVehicle by tiež použilo diagnostiku dronu a test pre tento experiment by zlyhal. Pri bežnom návrhu API najskôr rozhodnite, aké správanie má volajúci dostať.

U4 Auditná história

`string.Join` spojí položky zoznamu. Tu nejde o skutočnú bezpečnostnú kontrolu, ale o explicitný spôsob volania rozhrania.

```
return $"[INTERNÝ AUDIT] Balík {Package.TrackingNumber}, "  
+ $"História: {string.Join(" -> ", _auditTrail)}";
```

Volanie: `((ISecureAuditable)tracked).GetAuditRecord()`. Nemeňte podpis na public metódu triedy. História aktualizujte cez pripravené `UpdateLocation`.

Ako čítať zlyhanie testu

Otvorte prvý neúspešný test a porovnajte **Expected** s **Actual**. `NotImplementedException` znamená nedoplnené telo. Iná výnimka ukazuje problém, napríklad index mimo rozsahu. Chyba prekladu sa rieši pred spustením testov. Nesnažte sa opraviť výsledok zmenou očakávanej hodnoty v dodanom teste.

Pomoc pre depo a dispečing

U5 Indexer validácia a štatistiky

U5a môže používať pripravený LINQ namespace cez implicit usings.

```
return _packages.FirstOrDefault(p => string.Equals(
    p.TrackingNumber, trackingNumber,
    StringComparison.OrdinalIgnoreCase));
```

U5b zachová lokálne funkcie a vloží balík iba pri úspechu.

```
bool IsPrefixValid(string code) =>
    code.StartsWith(CodePrefix, StringComparison.OrdinalIgnoreCase)
    && code.Length >= minCodeLength;
static bool IsWeightValid(double weight) =>
    weight is > 0.01 and <= 100.0;
if (!IsPrefixValid(package.TrackingNumber)
    || !IsWeightValid(package.WeightKg)) return false;
_packages.Add(package);
return true;
```

U5c vráti n-ticu v poradí deklarovanom v návratovom type.

```
return (_packages.Count, _packages.Sum(p => p.BasePrice),
    _packages.Sum(p => p.WeightKg));
```

U6 Switch výraz a rozširujúca metóda

Tento vzor patrí za pripravenú validáciu v Dispatch.

```
return (package.Flags, distanceKm, package.WeightKg) switch
{
    (var f, _, _) when f.HasFlag(PackageFlags.RequiresColdChain)
        => ("Chladiarenská dodávka", package.BasePrice * 1.50m),
    (var f, <= 5, <= 8) when !f.HasFlag(PackageFlags.Heavy)
        && !f.HasFlag(PackageFlags.Oversized)
        => ("Elektrobicykel", package.BasePrice * 0.90m),
    (_, _, > 20) => ("Ťažká nákladná dodávka", package.BasePrice * 1.35m),
    (var f, _, _) when f.HasFlag(PackageFlags.Heavy)
        || f.HasFlag(PackageFlags.Oversized)
        => ("Ťažká nákladná dodávka", package.BasePrice * 1.35m),
    (_, > 60, _) => ("Diaľkový kamión", package.BasePrice * 1.25m),
    _ => ("Štandardná dodávka", package.BasePrice)
};
```

U6b musí počítat aj kombinované príznaky.

```
return packages.Where(p => p.Flags.HasFlag(PackageFlags.Express))
    .Sum(p => p.BasePrice);
```

Ďalšie vysvetlenie: Prednáška P02 Základy jazyka C# 2, snímky uvedené pri blokoch; Microsoft Learn: [hodnotové štruktúry](#), [rekordy a plytké kopírovanie](#), [explicitné rozhrania](#), [Copilot Chat](#), [režim Agent](#).